NMAM Institute of Technology, Nitte

Department of Computer Science and Engineering

C++ Programming Laboratory

LABORATORY MANUAL

Academic Year 2026–27

Name : Prajwal Jogi

USN : _____

Semester / Section : _____

INDEX

Expt. No.	Date	Title of the Experiment	Page No.	Signature
1	11/08/2026	BankAccount class	1	
2	18/08/2026	Time class: objects as arguments	3	
3	25/08/2026	Complex: overloading, friend function	5	
4	01/09/2026	Employee: array of objects	7	
5	08/09/2026	Sample: constructor overloading	9	
6	15/09/2026	Hierarchical inheritance	11	
7	29/09/2026	Operator overloading	13	
8	29/09/2026	Dynamic constructor (String class)	15	

Experiment No. 1

BankAccount class

Aim: Write a C++ program to design a class called `BankAccount`. Include the following data members like the name of depositor, account number and balance. Use the following member functions:

a) To initialize values

b) To deposit an amount

c) To withdraw an amount

d) To display name and balance

Program

```cpp
#include <iostream>
using namespace std;

class BankAccount
{
    private:
        int accno;
        string name;
        float balance;
        int amt;
    public:
        void read()
        {
            cout << "Enter name:";
            cin >> name;
            cout << "Enter account number:";
            cin >> accno;
            cout << "Enter balance:";
            cin >> balance;
        }
        void total_balance()
        {
            cout << "Current balance:" << balance << endl;
        }
        void deposit()
        {
            cout << "\nEnter amount to be deposited:";
            cin >> amt;
            balance = balance + amt;
            total_balance();
        }
        void withdraw()
        {
            cout << "\nEnter amount to be withdrawn:";
            cin >> amt;
            if (amt > balance)
            {
                cout << "Insufficient balance";
            }
            else
            {
                balance = balance - amt;
                total_balance();
            }
        }
        void display()
        {
```

```
            cout << "\nAccount Details:" << endl;
            cout << "Name of the depositor:" << name << endl;
            cout << "Account Number:" << accno << endl;
            cout << "Balance:" << balance << endl;
        }
};

int main()
{
    BankAccount b;
    b.read();
    b.display();
    b.deposit();
    b.withdraw();

    return 0;
}
```

Output

```
Enter name: Ravi
Enter account number: 10110
Enter balance: 50000

Account Details:
Name of the depositor: Ravi
Account Number: 10110
Balance: 50000

Enter amount to be deposited: 5000
Current balance: 55000

Enter amount to be withdrawn: 6000
Current balance: 49000


Enter name: Jitesh
Enter account number: 10111
Enter balance: 40000

Account Details:
Name of the depositor: Jitesh
Account Number: 10111
Balance: 40000

Enter amount to be deposited: 2000
Current balance: 42000

Enter amount to be withdrawn: 45000
Insufficient balance
```

Experiment No. 2

Time class: objects as arguments

Aim: Write a C++ program with class `Time` with data members that represents hours and minutes. Include appropriate member functions to compute time in hours and minutes (Use of objects as arguments).

Program

```cpp
#include <iostream>
using namespace std;

class Time
{
    private:
        int hour, minutes;
    public:
        void read()
        {
            cout << "Enter hours:";
            cin >> hour;
            cout << "Enter minutes:";
            cin >> minutes;
        }
        void display()
        {
            cout << "\nThe Time is: " << hour << " hours and " << minutes
                 << " minutes" << endl;
        }
        void compute(Time t1, Time t2)
        {
            int h = (t1.minutes + t2.minutes) / 60;
            minutes = (t1.minutes + t2.minutes) % 60;
            hour = t1.hour + t2.hour + h;
        }
};

int main()
{
    Time A, B, C;
    cout << "\nA" << endl;
    A.read();
    cout << "\nB" << endl;
    B.read();
    cout << "\nA" << endl;
    A.display();
    cout << "\nB" << endl;
    B.display();
    C.compute(A, B);
    cout << "\nC" << endl;
    C.display();
    return 0;
}
```

Output

```
A
Enter hours: 4
Enter minutes: 40

B
Enter hours: 4
Enter minutes: 40
```

```
A
The Time is: 4 hours and 40 minutes

B
The Time is: 4 hours and 40 minutes

C
The Time is: 9 hours and 20 minutes
```

Experiment No. 3

Complex: overloading, friend function

Aim: Write a C++ program to create a class called `Complex` and implement the following overloading function `ADD` that returns a complex number:

i) `ADD(a, c2)` where `a` is an integer and `c2` is a complex number

ii) `ADD(c1, c2)` where `c1` and `c2` are complex numbers.

Use function overloading and friend function concept for the implementation.

Program

```cpp
#include <iostream>
using namespace std;

class Complex {
    private:
        int r, i;
    public:
        void read();
        void display();
        friend Complex ADD(int x, Complex c);
        friend Complex ADD(Complex c1, Complex c2);
};

void Complex::read() {
    cout << "Enter Real number:";
    cin >> r;
    cout << "Enter Imaginary number:";
    cin >> i;
}

void Complex::display() {
    cout << r << "+" << i << "i";
}

Complex ADD(int x, Complex c) {
    Complex t;
    t.r = x + c.r;
    t.i = c.i;
    return t;
}

Complex ADD(Complex c1, Complex c2) {
    Complex t;
    t.r = c1.r + c2.r;
    t.i = c1.i + c2.i;
    return t;
}

int main() {
    Complex c1, c2, c3, c4;
    int a;
    cout << "Enter 1st complex number:";
    c1.read();
    cout << "Enter 2nd complex number:";
    c2.read();
    cout << "Enter value of a:";
    cin >> a;
    c3 = ADD(a, c2);
    c4 = ADD(c1, c2);
```

```
50      cout << "1st Complex number:";
51      c1.display();
52      cout << "2nd Complex number:";
53      c2.display();
54      cout << "ADD(a,c2):";
55      c3.display();
56      cout << "ADD(c1,c2):";
57      c4.display();
58      return 0;
59  }
```

Output

```
Enter 1st complex number:
Enter Real number:
3
Enter Imaginary number:
4
Enter 2nd complex number:
Enter Real number:
2
Enter Imaginary number:
1
Enter value of a: 1
1st Complex number:
3+4i
2nd Complex number:
2+1i
ADD(a,c2):
3+1i
ADD(c1,c2):
5+5i
```

Experiment No. 4

Employee: array of objects

Aim: Given that an `Employee` class contains the following members (data members are employee name and salary, member functions are read and print). Write a C++ program to read the data of *n* employees and display the details of each employee (use array of objects concept).

Program

```cpp
#include <iostream>
using namespace std;

class Employee
{
    private:
        int eno;
        string ename;
        double salary;
    public:
        void readdata()
        {
            cout << "Enter employee number:";
            cin >> eno;
            cout << "Enter employee name:";
            cin >> ename;
            cout << "Enter employee salary:";
            cin >> salary;
        }
        void displaydata()
        {
            cout << "Employee number:" << eno << endl;
            cout << "Employee name:" << ename << endl;
            cout << "Employee salary:" << salary << endl;
        }
};

int main()
{
    int n;
    Employee e[10];
    cout << "Enter the number of employees:" << endl;
    cin >> n;
    for(int i=0; i<n; i++)
    {
        e[i].readdata();
        cout << endl;
    }
    for(int i=0; i<n; i++)
    {
        e[i].displaydata();
        cout << endl;
    }
    return 0;
}
```

Output

```
Enter the number of employees:
3
Enter employee number: 521
Enter employee name: Prajwal
Enter employee salary: 70000.99

Enter employee number: 522
Enter employee name: Jogi
Enter employee salary: 80000.99

Enter employee number: 523
Enter employee name: Jitesh
Enter employee salary: 90000.99

Employee number: 521
Employee name: Prajwal
Employee salary: 70000.99

Employee number: 522
Employee name: Jogi
Employee salary: 80000.99

Employee number: 523
Employee name: Jitesh
Employee salary: 90000.99
```

Experiment No. 5

Sample: constructor overloading

Aim: Write a C++ program to create a class `Sample` with integer, character and float data members to determine a constructor overloading on this class with all types of constructors including default argument constructor.

Program

```cpp
#include <iostream>
using namespace std;

class Sample
{
    int i;
    char c;
    double d;
    public: Sample()
    {
        i = 0;
        c = '\0';
        d = 0.0;
    }
    Sample(int x, char y, double z)
    {
        i = x;
        c = y;
        d = z;
    }
    Sample(Sample &s)
    {
        i = s.i;
        c = s.c;
        d = s.d;
    }
    Sample(int x, double z, char y='s')
    {
        i = x;
        c = y;
        d = z;
    }
    void display()
    {
        cout << "Integer:" << i << endl;
        cout << "Character:" << c << endl;
        cout << "Float:" << d << endl;
    }
};

int main()
{
    Sample S1;
    cout << "S1";
    S1.display();
    Sample S2(10, 'a', 4.5);
    cout << "S2";
    S2.display();
    Sample S3(30, 4.5);
    cout << "S3";
    S3.display();
    Sample S4(S2);
    cout << "S4";
```

```
54      S4.display();
55      return 0;
56  }
```

Output

```
S1
Integer: 0
Character:
Float: 0

S2
Integer: 10
Character: a
Float: 4.5

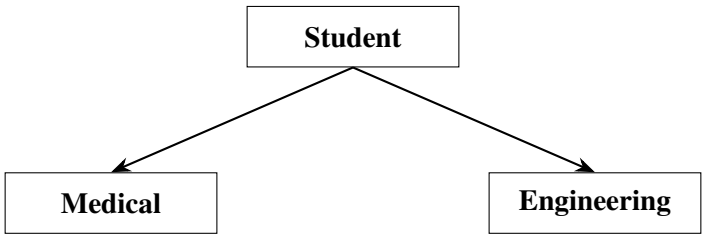
S3
Integer: 30
Character: s
Float: 4.5

S4
Integer: 10
Character: a
Float: 4.5
```

Experiment No. 6

Hierarchical inheritance

Aim: Write a C++ program to create and implement classes as shown in the figure using hierarchical inheritance.



Program

```cpp
#include <iostream>

using namespace std;

class Student
{
    protected: char name[20];
        int age, usn;
    public:
        void getStudent()
        {
            cout << "Enter name, age, usn\n";
            cin >> name >> age >> usn;
        }
};

class Medical: public Student
{
    int year;
    public:
        void getMedical()
        {
            cout << "Year:";
            cin >> year;
        }
        void display()
        {
            cout << "\n Medical Student details:\n";
            cout << " Age:" << age;
            cout << "\nName:" << name;
            cout << "\nUSN:" << usn;
            cout << "\nYear:" << year;
        }
};

class Engineering: public Student
{
    int sem;
    char branch[3];
    public:
        void getEngineering()
        {
            cout << "Enter sem, branch:\n";
            cin >> sem >> branch;
```

```
        }
        void display()
        {
            cout << "\n Engineering Student details:\n";
            cout << "Name:" << name;
            cout << "\nAge:" << age;
            cout << "\nUSN:" << usn;
            cout << "\nSem:" << sem;
            cout << "\nBranch:" << branch;
        }
};

int main()
{
    Medical m1;
    Engineering e1;
    cout << "Enter Medical Student details:\n";
    m1.getStudent();
    m1.getMedical();
    cout << "Enter Engineering Student details:\n";
    e1.getStudent();
    e1.getEngineering();
    m1.display();
    e1.display();
    return 0;
}
```

Output

```
Enter Medical Student details:
Enter name, age, usn:
Ravi
24
524
Year: 4

Enter Engineering Student details:
Enter name, age, usn:
Sahan
24
525
Enter sem, branch:
5
CSE

Medical Student details:
Name: Ravi
Age: 24
USN: 524
Year: 4

Engineering Student details:
Name: Sahan
Age: 24
USN: 525
Sem: 5
Branch: CSE
```

Experiment No. 7

Operator overloading

Aim: Write a C++ program to overload binary + and − operator to add and subtract two complex numbers. Define relevant data members for reading and displaying complex numbers.

Program

```cpp
#include <iostream>
using namespace std;

class Complex
{
    int real;
    float imag;
    public:
        void read(int r, float i)
        {
            real = r;
            imag = i;
        }
        Complex operator + (Complex c)
        {
            Complex temp;
            temp.real = real + c.real;
            temp.imag = imag + c.imag;
            return temp;
        }
        Complex operator - (Complex c)
        {
            Complex temp;
            temp.real = real - c.real;
            temp.imag = imag - c.imag;
            return temp;
        }
        void display()
        {
            cout << "Real: " << real << endl;
            cout << "Imaginary:" << imag << endl;
            cout << real << "+" << imag << "i" << endl;
        }
};

int main()
{
    Complex c1, c2, c3, c4;
    c1.read(4, 5.5);
    c2.read(3, 1.5);
    cout << "C1" << endl;
    c1.display();
    cout << "C2" << endl;
    c2.display();
    c3 = c1 + c2;
    c4 = c1 - c2;
    cout << "C3 = C1 + C2" << endl;
    c3.display();
    cout << "C4 = C1 - C2" << endl;
    c4.display();
    return 0;
}
```

Output

```
C1
Real: 4
Imaginary: 5.5
4+5.5i

C2
Real: 3
Imaginary: 1.5
3+1.5i

C3 = C1 + C2
Real: 7
Imaginary: 7
7+7i

C4 = C1 - C2
Real: 1
Imaginary: 4
1+4i
```

Experiment No. 8

Dynamic constructor (String class)

Aim: Write a C++ program to demonstrate the working of dynamic constructors using the class `String`, with string as the data members inside the class, include appropriate member functions to display the object data and a member function to concatenate two strings.

Program

```cpp
#include <iostream>
#include <cstring>
using namespace std;

class String
{
    int length;
    char *name;
    public:
        String()
        {
        }
        String(char s[])
        {
            length = strlen(s);
            name = new char[length + 1];
            strcpy(name, s);
        }
        void join(String A, String B)
        {
            length = A.length + B.length;
            name = new char[length + 1];
            name = strcpy(name, A.name);
            name = strcat(name, B.name);
        }
        void display()
        {
            cout << "name:" << name << endl;
        }
};

int main()
{
    String S1("Wel");
    String S2("fare");
    cout << "S1" << endl;
    S1.display();
    cout << "S2" << endl;
    S2.display();
    String S3;
    S3.join(S1, S2);
    String S4;
    S4.join(S1, S3);
    cout << "S3" << endl;
    S3.display();
    cout << "S4" << endl;
    S4.display();
    return 0;
}
```

Output

```
S1
name: Wel

S2
name: fare

S3
name: Welfare

S4
name: WelWelfare
```